

DSA Fundamentals

Placement Training Program

Arrays & Strings, Revisited for Problem-Solving

In a DSA context, arrays and strings are the raw material almost every coding interview problem starts from. Common patterns worth internalizing: the two-pointer technique (using two indices moving through an array, often from opposite ends, to solve problems in one pass instead of nested loops), and the sliding window technique (maintaining a moving subrange of an array/string to solve problems about contiguous subsequences efficiently). Recognizing which pattern a problem is asking for — often signaled by phrases like 'contiguous subarray' or 'pair that sums to X' — is a large part of solving array/string problems quickly under interview time pressure.

Linked Lists

A linked list stores elements as a chain of nodes, each holding a value and a reference to the next node, rather than in one contiguous block of memory like an array. This makes insertion and deletion at a known position $O(1)$ (just relinking pointers) instead of an array's $O(n)$ (shifting every subsequent element), at the cost of losing $O(1)$ random access by index — you have to walk from the head to reach any given position. Classic linked list interview problems (reversing a list, detecting a cycle, finding the middle element) are really exercises in careful pointer manipulation, and drawing the pointers out on paper before coding is a genuinely effective technique, not just a beginner's crutch.

Stacks & Queues

A stack is Last-In-First-Out (LIFO) — the most recently added element is the first removed, like a stack of plates. A queue is First-In-First-Out (FIFO) — elements are removed in the order they were added, like a line of people. Stacks show up naturally in problems involving nested/matching structures (balanced parentheses, undo functionality, function call tracking) and in implementing depth-first search. Queues show up in problems involving processing order (task scheduling) and are the natural structure for breadth-first search, where you need to explore nodes level by level rather than depth by depth.

Recursion

A recursive function solves a problem by calling itself on a smaller version of the same problem, with a base case that stops the recursion. Every recursive solution needs both pieces clearly defined: what's the smallest version of the problem that can be answered directly (the base case), and how does solving a smaller version help solve the larger one (the recursive case)? Recursion maps naturally onto problems with a self-similar, tree-like structure (traversing trees,

generating combinations/permutations, divide-and-conquer algorithms) — trying to force those same problems into a purely iterative solution is often far more complex than the recursive version.

Time/Space Complexity

Big-O notation describes how an algorithm's running time or memory use grows as input size grows, ignoring constant factors — it answers 'how does this scale,' not 'exactly how many milliseconds does this take.' Common complexities, from best to worst: $O(1)$ constant, $O(\log n)$ logarithmic (like binary search), $O(n)$ linear, $O(n \log n)$ (like efficient sorting), $O(n^2)$ quadratic (like a naive nested-loop comparison), $O(2^n)$ exponential (like naive recursive Fibonacci without memoization). Being able to look at a piece of code — especially nested loops and recursive calls — and state its Big-O complexity is one of the most consistently tested skills in technical interviews, precisely because it reveals whether a candidate understands why a solution will or won't scale.

Learn More

GeeksforGeeks — Data Structures

<https://www.geeksforgeeks.org/data-structures/>

NeetCode — DSA Roadmap & Practice

<https://neetcode.io/>

Visualgo — Visualizing Data Structures & Algorithms

<https://visualgo.net/>